

Capstone Project Guide

End-to-End Insurance Claims Automation Platform
LangChain + RAG + LangGraph Multi-Agent + MCP + FastAPI +
LangSmith + Azure Deployment
Step-by-Step Beginner-Friendly Implementation Workbook

Purpose

This guide helps participants build, test, package and deploy an enterprise-style Insurance Claims Automation Platform on Azure Cloud. It starts from project environment creation and ends with testing the deployed project URL.

Item	Description
Audience	UiPath Developers, RPA Developers, Automation Leads and Solution Architects
Domain	Insurance Claims, Policy Servicing, Fraud Review, Human Review and Audit
Core Stack	FastAPI, Python, LangGraph, LangChain, RAG, MCP-style tools, LangSmith and Azure
Deployment	Docker image in Azure Container Registry, running on Azure App Service
Integration	UiPath HTTP Request activity and enterprise REST API calls

Table of Contents

1. Capstone Objective and Business Outcome
2. End-to-End Architecture
3. Prerequisites and VS Code Setup
4. Project Folder Structure
5. API Keys and LangSmith Setup
6. Python Dependencies and requirements.txt
7. LangGraph Multi-Agent Workflow
8. FastAPI API Layer
9. Local Testing with Swagger UI
10. Docker Image Creation and Local Container Test
11. Azure Deployment Architecture
12. Azure CLI Deployment Commands
13. Azure Environment Variables and Key Vault
14. Test Deployed Project Link
15. UiPath Integration
16. LangSmith Debugging
17. Update, Rollback and Logs
18. Security, Governance and Cleanup
19. Final Demo Script and Checklist
20. References

1. Capstone Objective and Business Outcome

The objective is to build and deploy an enterprise-ready Insurance Claims Automation Platform. The platform accepts claim data, validates information, uses RAG to answer policy questions, uses Generative AI to summarize claim context, and uses LangGraph agents to route the claim.

- Accept claim documents or claim data from UiPath, portals or APIs.
- Extract and validate structured claim information using Python.
- Use RAG to answer policy and coverage questions using retrieved policy clauses.
- Use LangGraph multi-agent flow for policy verification, document validation, risk review, routing and human review.
- Expose MCP-style tools for policy lookup, claim validation and document analysis.
- Provide FastAPI endpoints for UiPath and enterprise systems.
- Use LangSmith for tracing and debugging AI workflows.
- Deploy securely on Azure Cloud.

Beginner Explanation

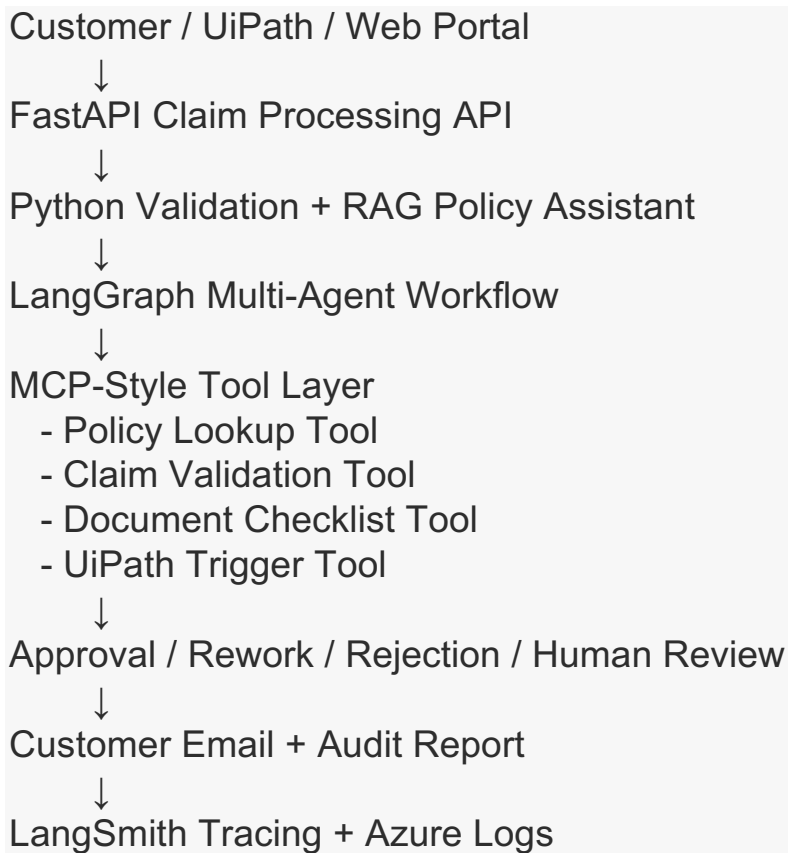
This project turns your local AI claim-processing code into a cloud service. After deployment, UiPath can send claims to a secure URL and receive a JSON response such as Approved, Rework Required, Human Review or Policy Exception.

Step	Workflow Activity
1	Customer uploads claim document or submits claim data
2	Python extracts and validates structured data
3	Generative AI summarizes claim and policy context
4	Policy verification agent checks coverage
5	Fraud review agent identifies risk indicators
6	LangGraph routes to approval, rejection or human review
7	MCP-style tools expose policy lookup and claim validation
8	UiPath robot executes downstream

	automation
9	System generates customer email and audit report

2. End-to-End Architecture

The architecture uses FastAPI as the API layer, LangGraph as the orchestration layer, RAG for policy knowledge, MCP-style tools for business actions, LangSmith for tracing and Azure App Service for hosting.



Layer	Responsibility	Simple Meaning
FastAPI	Expose REST endpoints	The front desk URL for UiPath and external systems
LangGraph	Run multi-agent workflow	Controls claim steps and routing decisions
RAG	Retrieve policy context	Finds policy clauses before answering

MCP Tools	Expose approved business functions	Allows AI to call controlled tools
LangSmith	Trace and debug	Shows what happened inside AI workflow
Azure App Service	Host cloud API	Runs the app as a scalable cloud service

3. Prerequisites and VS Code Setup

Participants should prepare all required tools before writing code. This avoids deployment issues later.

Category	Requirement
Laptop	16 GB RAM recommended, stable internet
Python	Python 3.11 or above
Editor	Visual Studio Code
Docker	Docker Desktop installed and running
Azure	Azure subscription and Azure CLI
LLM Access	OpenRouter or Azure OpenAI compatible API key
Tracing	LangSmith API key, optional but recommended
UiPath	UiPath Studio for HTTP Request testing

Verify Tools

```
python --version
pip --version
az --version
docker --version
code --version
```

Create VS Code Project

```
mkdir insurance-claims-capstone
cd insurance-claims-capstone
code .
python -m venv .venv

# Windows
.venv\Scripts\activate

# macOS / Linux

source .venv/bin/activate
```

Trainer Tip

Confirm that the terminal shows the virtual environment before installing packages. This avoids conflicts with other Python installations.

4. Project Folder Structure

A clean folder structure helps participants understand the separation between API code, agent code, tools, RAG logic, deployment files and test data.

```
insurance-claims-capstone/
├── app.py
├── langgraph_agent.py
├── rag_service.py
├── mcp_tools.py
├── uipath_service.py
├── requirements.txt
├── Dockerfile
├── .env
├── .gitignore
├── README.md
├── data/
│   ├── sample_claims.json
│   └── policy_clauses.json
├── tests/
│   └── test_payloads.json
```

File	Purpose
app.py	FastAPI application and API endpoints
langgraph_agent.py	Multi-agent insurance claim workflow
rag_service.py	Policy retrieval and RAG answer generation
mcp_tools.py	Policy lookup and claim validation tools
uipath_service.py	Mock or real UiPath robot trigger
Dockerfile	Container build instructions
.env	Local environment variables; never commit to GitHub

5. API Keys and LangSmith Setup

API keys are sensitive. During local development, keep them in .env. During cloud deployment, use Azure App Settings or Azure Key Vault.

```
OPENROUTER_API_KEY=your_openrouter_api_key_here  
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1  
OPENROUTER_MODEL=openai/gpt-4.1-mini
```

```
LANGSMITH_TRACING=true  
LANGSMITH_API_KEY=your_langsmith_api_key_here  
LANGSMITH_PROJECT=insurance-claims-capstone
```

Create .gitignore

```
.env  
.venv/  
__pycache__/  
*.pyc  
*.log  
.DS_Store
```

Security Rule

Never place API keys in Python code, notebooks, Dockerfile, PowerPoint, GitHub or training screenshots.

6. Python Dependencies and requirements.txt

requirements.txt tells Python and Docker which libraries are required. It makes local setup and cloud deployment repeatable.

```
fastapi  
uvicorn  
python-dotenv  
openai  
langchain  
langchain-openai  
langgraph
```



```
langsmith
pydantic
requests
pandas
numpy
scikit-learn
chromadb
```

Install Locally

```
pip install -r requirements.txt
```

- FastAPI creates APIs.
- Uvicorn runs the FastAPI server.
- OpenAI client calls the LLM endpoint.
- LangGraph runs the multi-agent workflow.
- LangSmith traces AI workflow execution.
- Pandas and scikit-learn support data processing and RAG examples.

7. LangGraph Multi-Agent Workflow

LangGraph helps build a stateful workflow where each node is an agent. In this capstone, each agent performs one insurance responsibility.

State and LLM Setup

```
import os, json
from typing import TypedDict, Dict, Any, List
from openai import OpenAI
from langgraph.graph import StateGraph, START, END

client = OpenAI(
    base_url=os.getenv("OPENROUTER_BASE_URL"),
    api_key=os.getenv("OPENROUTER_API_KEY")
)

def call_llm(prompt: str) -> str:
    response = client.chat.completions.create(
        model=os.getenv("OPENROUTER_MODEL"),
        messages=[
```

```

{"role": "system", "content": "You are an insurance claims automation
assistant."},
{"role": "user", "content": prompt}
],
temperature=0.2
)
return response.choices[0].message.content

```

```

class ClaimState(TypedDict, total=False):
    claim: Dict[str, Any]
    claim_summary: str
    policy_status: str
    document_status: str
    risk_level: str
    route: str
    final_message: str
    logs: List[str]

```

Agent Responsibilities

Agent	Responsibility	Output
Claim Intake Agent	Summarize incoming claim	claim_summary
Policy Verification Agent	Check policy, premium and coverage	policy_status
Document Validation Agent	Check required and submitted documents	document_status
Fraud Risk Agent	Identify risk level	risk_level
Routing Agent	Select approval, rework or review path	route

8. Agent Nodes and Graph Compilation

The following compact sample shows how to create nodes and compile the graph. Participants can extend this with RAG, MCP tools and human review.

```
def claim_intake_agent(state):
    claim = state["claim"]
    summary = call_llm("Summarize this insurance claim: " + json.dumps(claim))
    logs = state.get("logs", []) + ["Claim intake completed"]
    return {**state, "claim_summary": summary, "logs": logs}

def routing_agent(state):
    if state.get("policy_status") != "Valid":
        route = "Policy Exception"
    elif state.get("document_status") == "Missing Documents":
        route = "Rework Required"
    elif state.get("risk_level") == "High":
        route = "Human Review"
    else:
        route = "Approved for Processing"
    return {**state, "route": route, "final_message": "Claim routed to: " + route}

workflow = StateGraph(ClaimState)
workflow.add_node("claim_intake", claim_intake_agent)
workflow.add_node("routing", routing_agent)
workflow.add_edge(START, "claim_intake")
workflow.add_edge("claim_intake", "routing")
workflow.add_edge("routing", END)
claim_agent_app = workflow.compile()
```

Practical Note

In the final project, add policy verification, document validation and fraud review between claim_intake and routing. Each node should add logs for auditability.

9. FastAPI API Layer

FastAPI exposes the LangGraph workflow as a REST API. UiPath or any enterprise system can call this endpoint.

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List, Optional
from langgraph_agent import process_claim_with_agent

app = FastAPI(title="Insurance LangGraph Agent API")

class ClaimRequest(BaseModel):
    claim_id: str
    policy_number: str
    customer_name: str
    claim_type: str
    claim_amount: float
    claim_description: Optional[str] = ""
    submitted_documents: List[str]

@app.get("/health")
def health_check():
    return {"status": "healthy"}

@app.post("/claim/process")
def process_claim(request: ClaimRequest):
    result = process_claim_with_agent(request.dict())
    return {"status": "success", "result": result}
```

10. Local Testing with Swagger UI

Always test locally before deploying to Azure. This reduces cloud troubleshooting time.

```
uvicorn app:app --reload --port 8000
```

Open the following URL in the browser:

```
http://127.0.0.1:8000/docs
```

Sample Claim Request

```
{
  "claim_id": "CLM9001",
  "policy_number": "POL1001",
  "customer_name": "Rajesh Sharma",
  "claim_type": "Health",
  "claim_amount": 85000,
  "claim_description": "Hospitalization due to dengue fever.",
  "submitted_documents": [
    "Hospital Bill",
    "Discharge Summary",
    "Doctor Prescription",
    "Claim Form"
  ]
}
```

Expected Business Output

```
{
  "policy_status": "Valid",
  "document_status": "Complete",
  "risk_level": "Low",
  "route": "Approved for Processing"
}
```

11. Docker Image Creation and Local Container Test

A Docker image packages the application, dependencies, Python runtime and startup command. This makes deployment consistent across laptop, Azure and client environments.

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 8000
```

```
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Build and Test Container

```
docker build -t insurance-langgraph-agent .
```

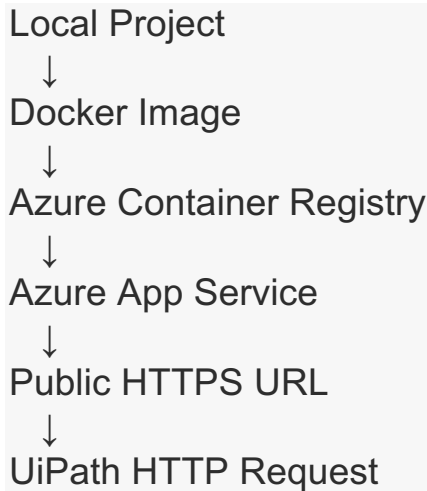
```
docker run -p 8000:8000 --env-file .env insurance-langgraph-agent
```

Checkpoint

Open <http://127.0.0.1:8000/docs> again. Continue only if the Docker container works locally.

12. Azure Deployment Architecture

The recommended beginner deployment path is Docker image to Azure Container Registry, then Azure App Service pulls and runs that image.



Azure Resource	Purpose
Resource Group	Holds all project resources
Azure Container Registry	Stores Docker image
App Service Plan	Defines compute and pricing tier
Web App	Runs the FastAPI container
App Settings	Runtime configuration and environment variables
Key Vault	Production-grade secret storage

13. Azure CLI Deployment Commands

Run these commands from the project folder after Docker testing is successful. Replace names if they are already taken globally.

Login and Create Resource Group

```

az login
az account show
az group create --name rg-insurance-agent-demo --location centralindia
  
```

Create Azure Container Registry

```
az acr create --resource-group rg-insurance-agent-demo --name  
insuranceagentacr --sku Basic
```

```
az acr login --name insuranceagentacr
```

Tag and Push Docker Image

```
docker tag insurance-langgraph-agent insuranceagentacr.azurecr.io/insurance-  
langgraph-agent:v1
```

```
docker push insuranceagentacr.azurecr.io/insurance-langgraph-agent:v1
```


14. Azure App Service Deployment Commands

Create App Service Plan

```
az appservice plan create --name asp-insurance-agent-demo --resource-group rg-insurance-agent-demo --is-linux --sku B1
```

Create Web App from Docker Image

```
az webapp create --resource-group rg-insurance-agent-demo --plan asp-insurance-agent-demo --name insurance-langgraph-agent-demo --deployment-container-image-name insuranceagentacr.azurecr.io/insurance-langgraph-agent:v1
```

Set Container Port

```
az webapp config appsettings set --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo --settings WEBSITES_PORT=8000
```

WEBSITES_PORT tells Azure that the FastAPI container listens on port 8000.

15. Configure Azure Environment Variables

Azure App Settings become environment variables inside the running container.

Your Python code reads them using `os.getenv()`.

```
az webapp config appsettings set --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo --settings OPENROUTER_BASE_URL="https://openrouter.ai/api/v1" OPENROUTER_MODEL="openai/gpt-4.1-mini" LANGSMITH_TRACING="true" LANGSMITH_PROJECT="insurance-claims-capstone"
```

Training Demo Key Setup

```
az webapp config appsettings set --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo --settings
```

```
OPENROUTER_API_KEY="your_openrouter_api_key_here" LANGSMITH_API_KEY="your_langsmith_api_key_here"
```

Production Reminder

For production, avoid direct secret values in command history. Use Azure Key Vault references.

16. Azure Key Vault for Production Secrets

Azure Key Vault stores secrets securely. App Service can use a managed identity to read secrets at runtime.

```
az keyvault create --name kv-insurance-agent-demo --resource-group rg-insurance-agent-demo --location centralindia
```

```
az keyvault secret set --vault-name kv-insurance-agent-demo --name OPENROUTER-API-KEY --value "your_openrouter_api_key_here"
```

```
az webapp identity assign --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo
```

Grant Web App Permission

```
az webapp identity show --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo --query principalId --output tsv
```

```
az keyvault set-policy --name kv-insurance-agent-demo --object-id YOUR_WEBAPP_PRINCIPAL_ID --secret-permissions get list
```

Configure Key Vault Reference

```
az webapp config appsettings set --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo --settings OPENROUTER_API_KEY="@Microsoft.KeyVault(SecretUri=https://kv-insurance-agent-demo.vault.azure.net/secrets/OPENROUTER-API-KEY/SECRET_VERSION)"
```

17. Restart and Test Deployed Project Link

```
az webapp restart --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo
```

```
az webapp browse --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo
```

Open the Swagger UI link:

<https://insurance-langgraph-agent-demo.azurewebsites.net/docs>

Test Using curl

```
curl https://insurance-langgraph-agent-demo.azurewebsites.net/health
```

```
curl -X POST https://insurance-langgraph-agent-  
demo.azurewebsites.net/claim/process -H "Content-Type: application/json" -d '{  
  "claim_id": "CLM9001",  
  "policy_number": "POL1001",  
  "customer_name": "Rajesh Sharma",  
  "claim_type": "Health",  
  "claim_amount": 85000,  
  "claim_description": "Hospitalization due to dengue fever.",  
  "submitted_documents": [  
    "Hospital Bill",  
    "Discharge Summary",  
    "Doctor Prescription",  
    "Claim Form"  
  ]  
}'
```

18. UiPath Integration

UiPath can invoke the Azure-deployed AI agent using HTTP Request activity.

UiPath Setting	Value
Method	POST
Endpoint	https://insurance-langgraph-agent-demo.azurewebsites.net/claim/process
Header	Content-Type: application/json
Body Type	JSON
Output	Deserialize JSON response and read result.route

UiPath Decision Logic

```

If route = "Approved for Processing"
    Continue downstream claim automation
Else If route = "Rework Required"
    Create missing document task
Else If route = "Human Review"
    Create Action Center or manual review task
Else
    Route to policy exception queue
  
```

Practical Tip

Use structured JSON fields like route, policy_status, document_status and risk_level. UiPath should not depend on parsing long AI-generated paragraphs.

19. LangSmith Tracing and Debugging

LangSmith helps trace the AI workflow. It is useful when there are multiple agents, prompts, tool calls and routing decisions.

- See the input sent to the workflow.
- Trace which LangGraph nodes executed.
- Inspect prompts and LLM outputs.
- Debug failed or unexpected decisions.
- Compare outputs across versions.
- Support audit and governance discussions.

LANGSMITH_TRACING=true

LANGSMITH_API_KEY=your_langsmith_api_key_here

LANGSMITH_PROJECT=insurance-claims-capstone

Enterprise Value

For insurance automation, tracing is important because claim decisions must be explainable and reviewable.

20. Update, Rollback and Logs

Use Docker image tags to deploy new versions and roll back if needed.

Deploy Version 2

```
docker build -t insurance-langgraph-agent .
docker tag insurance-langgraph-agent insuranceagentacr.azurecr.io/insurance-langgraph-agent:v2
docker push insuranceagentacr.azurecr.io/insurance-langgraph-agent:v2

az webapp config container set --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo --container-image-name insuranceagentacr.azurecr.io/insurance-langgraph-agent:v2

az webapp restart --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo
```

Rollback to Version 1

```
az webapp config container set --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo --container-image-name insuranceagentacr.azurecr.io/insurance-langgraph-agent:v1
```

View Logs

```
az webapp log config --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo --docker-container-logging filesystem

az webapp log tail --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo
```

21. Troubleshooting

Issue	Likely Cause	Action
App does not start	Wrong port or missing dependency	Check WEBSITES_PORT and logs
LLM API error	Missing or invalid API key	Check app settings or Key Vault
Docker push fails	ACR login expired	Run az acr login again
Swagger not opening	Container failed to start	Use az webapp log tail
LangGraph import error	Missing package	Update requirements.txt and rebuild image
UiPath response issue	Wrong endpoint or JSON body	Test same request in Swagger

Key Vault not resolving	Managed identity permission missing	Check access policy and restart app
-------------------------	-------------------------------------	-------------------------------------

```
az webapp config appsettings list --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo
```

```
az webapp log tail --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo
```

22. Security, Governance and Cleanup

Control Area	Recommended Practice
Secrets	Use Azure Key Vault for API keys and client secrets
API Access	Protect APIs using API Management, Entra ID or API keys
PII Handling	Avoid logging unnecessary personal data
Human Review	Required for high-risk or high-value claims
Audit Logs	Store route, decision reason, risk level and policy status
LLM Output	Validate JSON before UiPath uses it
Tool Access	Expose only approved MCP-style tools
Deployment	Use version tags and rollback strategy

Stop or Delete Resources

```
az webapp stop --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo
```

```
az webapp start --resource-group rg-insurance-agent-demo --name insurance-langgraph-agent-demo
```

```
az group delete --name rg-insurance-agent-demo --yes --no-wait
```

Warning

Deleting the resource group removes all resources inside it. Use this only after the training demo is complete.

23. Final Capstone Demo Script

1. Show architecture and explain FastAPI, LangGraph, RAG, MCP tools, LangSmith and Azure.
2. Open Azure Swagger UI project link.
3. Submit a valid health claim and show approval route.
4. Submit a claim with missing documents and show rework route.
5. Submit a high-value or suspicious claim and show human review route.
6. Explain how UiPath consumes result.route from JSON.

7. Show LangSmith trace or explain where traces appear.
8. Explain security using App Settings and Key Vault.
9. Explain update and rollback using Docker image tags.

Demo Conclusion

This capstone demonstrates how traditional UiPath automation can evolve into AI Automation Engineering by combining Python, FastAPI, RAG, LangGraph agents, MCP-style tools, LangSmith tracing and Azure deployment.

24. Participant Completion Checklist

Task	Completed
Project folder created	☐
VS Code and virtual environment configured	☐
requirements.txt installed	☐
.env created locally	☐
LangGraph workflow created	☐
FastAPI endpoint created	☐
Local Swagger test completed	☐
Dockerfile created	☐
Docker image built and tested locally	☐
Resource group created	☐
Azure Container Registry created	☐
Docker image pushed to ACR	☐
Azure App Service created	☐
Environment variables configured	☐
Azure URL tested	☐
UiPath HTTP Request tested	☐
LangSmith tracing configured	☐
Security checklist reviewed	☐

25. References and Further Reading

Official references used for deployment and tracing guidance:

- Microsoft Learn: Run a custom container in Azure App Service - <https://learn.microsoft.com/en-us/azure/app-service/quickstart-custom-container>
-
- Microsoft Learn: Environment variables and app settings in Azure App Service - <https://learn.microsoft.com/en-us/azure/app-service/reference-app-settings>
-
- Microsoft Learn: Use Key Vault references as app settings - <https://learn.microsoft.com/en-us/azure/app-service/app-service-key-vault-references>
-
- Microsoft Learn: Push and pull images using Azure Container Registry - <https://learn.microsoft.com/en-us/azure/container-registry/container-registry-get-started-docker-cli>
-
- LangChain Docs: LangSmith observability quickstart - <https://docs.langchain.com/langsmith/observability-quickstart>
-
- LangChain Docs: Trace LangGraph applications with LangSmith - <https://docs.langchain.com/langsmith/trace-with-langgraph>

Final Note

Complete the guide sequentially. Do not deploy to Azure until the FastAPI app and Docker container are tested successfully on the local machine.